

CSC10006 – Introduction to Database

Stored Procedure & Function

TABLE OF CONTENTS

1 Objectives.....	1
2 Detailed Instructions.....	1
2.1 Stored Procedure.....	1
2.1.1 Introduction.....	1
2.1.2 Syntax.....	2
2.1.3 Example.....	5
2.2 Function.....	7
2.2.1 Introduction.....	7
2.2.2 Syntax.....	7
2.2.3 Example.....	7
3 In-Class Exercises.....	8
4 Homework.....	10

1 Objectives

After completing this practical session, students will be able to:

- Understand the techniques for writing the simple functions and stored procedures in SQL Server.

2 Detailed Instructions

2.1 Stored Procedure

2.1.1 Introduction

When we create an application with Microsoft SQL Server, the Transact-SQL programming language is the primary language for communication between the application and the SQL Server database. When we create programs with Transact-SQL, two main methods can be used to store and execute programs:

- We can store running script locally and create applications to send commands to SQL Server and process the results.
- We can store running script as stored procedures in SQL Server and create applications to call and execute stored procedures and process the results.

Characteristics of Stored-procedures in SQL Server:

- Accept input parameters and return values contained in the parameters to call procedures or process them accordingly.
- Contain program commands to execute operations in the database, including calling other procedures to execute.
- Return status values to call procedures or perform operations to indicate whether the execution is successful or failed; if failed, the reason for the failure.

Stored procedures differ from functions in that their return value is not contained in the name and they are not used directly in expressions. We must use Transact-SQL command **EXECUTE** to call stored procedures.

Stored procedures have advantages over locally stored Transact-SQL programs:

- Stored procedures allow for program adjustments: We can create a stored procedure once and store it in the database, and in the program, we can call it with any number of times. Stored procedures can be identified by someone who created them, and their changes are completely independent with the source code of the program.
- Stored procedures allow for faster execution: if the processing requires a large Transact-SQL source code segment or the execution is repetitive, stored procedures execute faster than executing a series of Transact-SQL commands. They are parsed and optimized during the first execution and a compiled version of them is stored in memory for subsequent calls, meaning that in subsequent executions, they do not need to be parsed and optimized again, but they will use the previously compiled results, which means that in subsequent executions, they will use the previously compiled results.
- Stored procedures can reduce network transmission congestion: Suppose a processing uses a large number of Transact-SQL commands, and the execution goes through each line of command, so executing stored procedures will be shorter, because when not executing them, we have to send the command stream over the network, which will lead to network congestion.
- Stored procedures can be used in machine security issues: because users can be assigned permissions to use these stored procedures, even if they are not allowed to execute them directly.

2.1.2 Syntax

A Stored procedure is defined including the following components:

- The name of the stored procedure
- Parameters
- The body of the stored procedure: including the Transact-SQL commands to execute the procedure.

A stored procedure can be created using the "**Create Procedure**" command, and can be modified using the "**Alter Procedure**" command, and can be deleted using the "**Drop Procedure**" command in the Transact-SQL statement.

Create procedure Syntax:

```
CREATE PROCEDURE procedure_name
    {@parameter data_type input/output } /*various input/output variables*/
AS
BEGIN
    [declare variables for processing]
    {Transact-SQL commands}
    [RETURN value -- Stored procedures can return a value or not]
END
```

Transact-SQL commands for programming:

```
-- 1. One line Comment

/*
    2. Multiple lines
    Comment
*/

/*3. Declare variables*/
DECLARE @parameter_name data_type

/*4. Assign values to variables*/
SET @parameter_name=value

SELECT @parameter_name=column, ...
FROM ...

/*5. Print messages to the screen*/
print N'Unicode message string'
```

/*6. Error message */

raiserror (N'Error message content ', 16, 1)

/*7. Branching commands */

IF (condition can use sub-query statements with **EXISTS** keyword)

BEGIN

{These Commands will run if the condition is met.

If there is only 1 command, there is no need for BEGIN ... END}

END

/*8. Branching commands with ELSE */

IF (condition can use sub-query statements with **EXISTS** keyword)

BEGIN

{These Commands will run if the condition is met.

If there is only 1 command, there is no need for BEGIN ... END}

END

ELSE

BEGIN

{These Commands if the condition is not met.

If there is only 1 command, there is no need for BEGIN ... END}

END

/*9. While loop (Note: FOR loop not supported) */

WHILE (condition)

BEGIN

{Commands if the condition is met.

If there is only 1 command, there is no need for BEGIN ... END}

END

Compilation: Choose the correct command to Create stored-procedure → F5

Call to Execution:

EXEC procedure_name --Stored-proc without parameters

EXEC procedure_name Para1_value, Para2_value, ... --Stored-proc with parameters

--Get the return value of the stored procedure:

EXEC @variable = procedure_name --Stored-proc without parameters

EXEC @variable = procedure_name Para1_value, Para2_value, ... --Stored-proc with parameters

2.1.3 Example

Example 1: Write a stored procedure to calculate the sum of 2 numbers

```
CREATE PROCEDURE sp_Tong
    @So1 int, @So2 int, @Tong int output
AS
SET @Tong = @So1 + @So2;

--Compile to create stored-procedure→F5

-- Execute to Check
Declare @Sum int
Exec sp_Tong 1, -2, @Sum output
Print @Sum
```

Example 2: Write a stored procedure to calculate the sum of even numbers from m to n

```
CREATE PROCEDURE sp_TongChanMN
    @m int, @n int
AS
BEGIN
    Declare @tong int
    Declare @i int
    Set @tong = 0
    Set @i = @m
    While (@i < @n)
    BEGIN
        IF (@i % 2 = 0)
            SET @tong = @tong + @i
        SET @i = @i + 1
    END
```

```
END
Print @tong
END

--Compile stored-procedure→F5

--Execute to Check
Exec sp_TongChanMN 1, 20
```

Example 3: Write a stored procedure to check the existence of a teacher by code

```
CREATE PROCEDURE sp_KiemTraGVTonTai
    @MaGV char(9)
AS
    IF ( EXISTS (SELECT * FROM GIAOVIEN WHERE MAGV=@MAGV) )
        Print N'Giáo viên tồn tại'
    ELSE
        Print N'Không tồn tại giáo viên ! ' + @MaGV

--Compile stored-procedure→F5

--Execute to Check
Exec sp_KiemTraGVTonTai '001'
```

Example 4: Write a stored procedure to output the list of teachers of a department:

```
CREATE PROCEDURE sp_DanhSachGiaoVien
    @MaBM char(9)
AS
    SELECT * FROM GIAOVIEN WHERE MABM=@MaBM

--Compile stored-procedure→F5

--Execute to Check
Exec sp_DanhSachGiaoVien 'HTTT'
```


2.2 Function

2.2.1 Introduction

In SQL Server, we can write functions and return values. The types of functions that can be written are as follows:

- Functions that return scalar values: varchar, int,
- Functions that return table values: inline table-valued

2.2.2 Syntax

Create function return scalar value:

```
CREATE FUNCTION function_name ( [@parameter_name parameter_data_type] )
RETURNS [return Data-type] /*Returns has 's' */
AS
Begin
    return ([scalar value])
End
```

Create function that returns a table:

```
CREATE FUNCTION function_name ( [@parameter_name parameter_data_type] )
RETURNS table
AS
    return [select command]
```

2.2.3 Example

Example 5: Write a function to calculate age based on date of birth

```
-- Create function
CREATE FUNCTION fTinhTuoi ( @ngaysinh datetime )
RETURNS int
AS
BEGIN
    return year(getdate()) – year(@ngaysinh)
END

-- Compile: F5
```

```
-- Execute to Check
print dbo.fTinhTuoi ('1/1/2000')
-- or
SELECT MAGV, dbo.fTinhTuoi (NgaySinh)
FROM GIAOVIEN
```

Example 6: Write a function to get the list of teachers of the HTTT department

```
-- Create function
CREATE FUNTION fDSGV_HTTT ( )
RETURNS table
AS
    return (SELECT * FROM GIAO VIEN WHERE MABM='HTTT')

-- Compile: F5

-- Execute to Check
SELECT * FROM dbo.fDSGV_HTTT ( )
```

3 In-Class Exercises

Duration: 02 hours.

Requirement:

Write the following stored procedures:

- Print the greeting "Hello World !!!".
- Print the sum of 2 numbers.
- Calculate the sum of 2 numbers (use an output variable to store the result).
- Print the sum of 3 numbers (reuse the Calculate sum of 2 numbers stored procedure).
- Print the sum of consecutive integers from m to n.
- Check if an integer is a prime number or not.

- g. Print the sum of prime numbers in the range m, n.
- h. Calculate the greatest common divisor of 2 integers.
- i. Calculate the least common multiple of 2 integers.

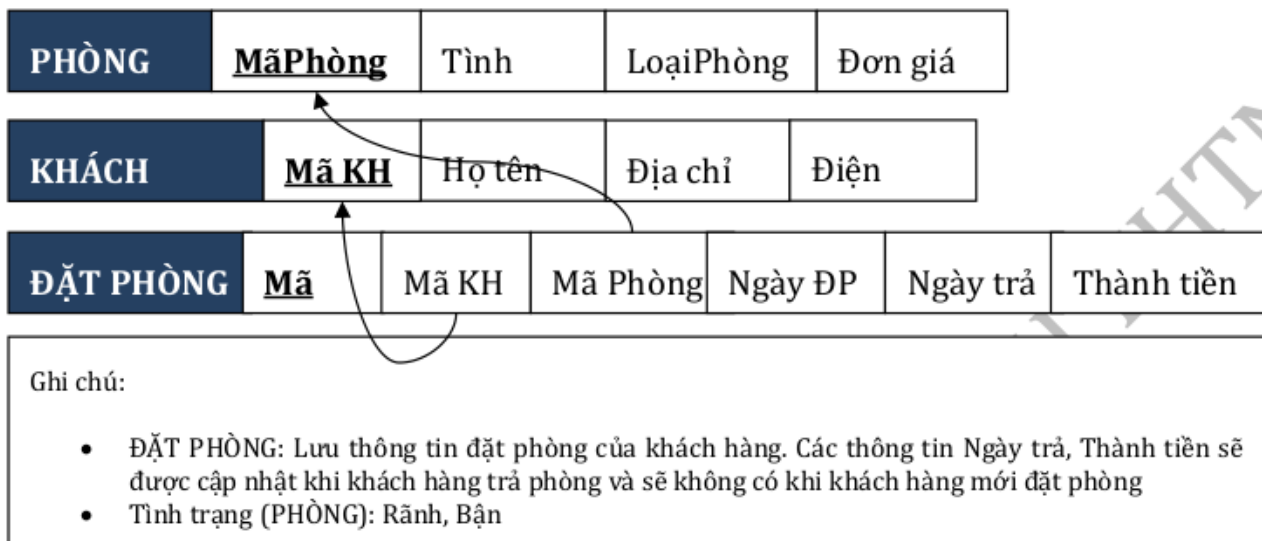
Write the following stored procedures:

- j. Output the entire list of teachers.
- k. Calculate the number of research projects a teacher is currently working on.
- l. Print detailed information of a teacher (using the print command): Personal information, Number of research projects participated in, Number of relatives of the teacher.
- m. Check if a teacher exists (based on MAGV).
- n. Check the regulations of a teacher: Only allowed to participate in research projects whose department the teacher is the head of.
- o. Perform an assignment for a teacher to work on a project:
 - Check input validity: teacher must exist, job must exist, participation time must be >0
 - Check the regulations in question n.
- p. Delete a teacher by code. If the teacher has related information (Has relatives, participates in research, ...), report an error.
- q. Print a list of teachers of a department along with the number of research projects they participate in, the number of relatives, the number of teachers they manage, ...
- r. Check the regulations of 2 teachers a, b: If a is the head of department c of b, then a's salary must be higher than b's salary. (a, b: teacher code)
- s. Add a teacher: Check the regulations: No duplicate names, age > 18, salary > 0
- t. Teacher code is automatically generated according to the rule: If there are teachers 001, 002, 003, then the MAGV of the new teacher will be 004. If there are teachers 001, 002, 005, then the MAGV of the new teacher will be 003.

4 Homework

Duration: 03 hours.

Let the database schema be:



Requirement:

1. Write the following stored procedure:

Name of stored procedure: **spDatPhong**

Content: record the booking information of the customer into the database.

Required parameters: customer code (@makh), room code (@maphong), booking date (@ngaydat).

Note: The booking code is an integer and must be generated automatically as follows: booking code = largest booking code + 1.

Other requirements and checks and calculations:

- Check that the customer code is valid (must appear in the KHÁCH HÀNG table)
- Room code valid (must appear in the PHÒNG table)
- Only be able to book a room when the room status is "**Rảnh**"
- If all checks are valid, record the booking information in the CSDL (Date of return and payment amount when booking are NULL)
- After booking is successful, the room status must be updated to "**Bận**"

Other requirements: Must have a verbal explanation of the steps involved.

2. Stored procedure/function

Name of stored procedure: **spTraPhong**

Content: record the check-out information of the customer into the database.

Required parameters: booking code (@madp), customer code (@makh)

Requirements for checks and calculations:

- Check the validity of the booking code, customer code: Valid if the customer has made a booking.
- Check-out date is today's date.
- Payment amount is calculated by the formula: Tien = Number of days borrowed x room price.
- The room status must be updated to "**Rảnh**" after recording the check-out information.

Other requirements: Must have a verbal explanation of the steps involved.